RAMOA Mind Map & Execution Order

basic mind map——-

```
                    ┌──────────────┐
                    │  Raw Logs    │
                    └──────┬───────┘
                           │
                  ┌────────▼─────────┐
                  │ Event Normalizer  │
                  └──────────┬────────┘
                             │
        ┌────────────────────┼────────────────────┐
        │                    │                    │
        ▼                    ▼                    ▼
  Knowledge Base     Heuristic Risk     Graph Layer
   (Cache Check)          Score           Score
        │                    │                    │
        └──────────────────┬─┴──┬─────────────────┘
                           ▼    ▼
                 Risk Combination
                           │
                 Non-linear Scaling
                           │
                  ┌────────▼────┐
                  │Orchestrator│
                  └────────────┘
                           │
                Select Agents
                           │
        ┌──────────────────┼──────────────────┐
        ▼                  ▼                  ▼
    Agent 1            Agent 2            Agent 3
                           │
                           ▼
                 Confidence Fusion
                           │
                           ▼
                 Final Decision
                           │
        ┌──────────────────┼──────────────────┐
        ▼                                     ▼
   Trust Update                     Store in Cache
```

DEPENDENCY LOGIC (What depends on what)

Level 1 – Independent Modules

These can be built separately:

- Event Normalizer
- Heuristic Risk
- Graph Layer
- Agents
- Knowledge Base

> None of these need each other.
> They only need the `Event` object.

Level 2 – Semi-Dependent

Risk Combination depends on:

- Heuristic Risk output
- Graph Score output

Orchestrator depends on:

- Scaled Risk
- Trust values
- Agent execution cost

Fusion depends on:

- Agent outputs
- Trust values

Trust update depends on:

- Final decision
- Agent correctness

Evaluation depends on:

- Everything working

ORDER TO BUILD (Correct Sequence)

Step 1 – Event Normalization

Build `Event` object.

> Everything else uses it.
> If this is unstable, everything breaks.

Step 2 – Build 1 Simple Agent

Just one **dummy agent** first.

Make sure:

`Event → Agent → returns confidence`

> Now you have analysis capability.

Step 3 – Build Heuristic Risk

Return simple float 0–1.

Test it independently.

Step 4 – Build Orchestrator Skeleton

For now:

- Hardcode trust
- Hardcode cost
- Use simple threshold

Just make:

`Risk → Select Agent → Run Agent`

> Now you have minimal RAMOA working.

Step 5 – Add Trust Engine

Implement:

- Trust initialization
- Trust update logic

Integrate with **orchestrator.**

> Now the brain evolves.

Step 6 – Add Graph Layer

Build graph logic separately.

Return `GraphAnomalyScore`.

Then plug into **risk combination.**

> Now **risk** becomes smarter.

Step 7 – Add Remaining Agents

Plug in:

- Network agent
- Statistical agent

> **Orchestrator** now selects among multiple agents.

Step 8 – Add Knowledge Base

Before risk step:

Check cache.

`If hit → return stored result`

Plug in carefully.

Step 9 – Add Evaluation Framework

Only now.

Run:

- Static mode (all agents)
- RAMOA mode (selective)

Compare metrics.

INTEGRATION AREAS

`3` **Main Integration Points:**

1. Risk Combination

   (Heuristic + Graph)

2. Orchestrator

   (Risk + Trust + Agent Cost)

3. Fusion

   (Agent outputs + Trust)

Everything else is modular.

Simplified Dependency Tree

Foundation:

- Event Normalization

First Floor:

- Heuristic Risk
- Agents
- Graph Layer

Second Floor:

- Risk Combination
- Orchestrator

Third Floor:

- Fusion
- Trust Update

Roof:

- Knowledge Base
- Evaluation